Matplotlib

is a low level graph plotting library in python that serves as a visualization utility.

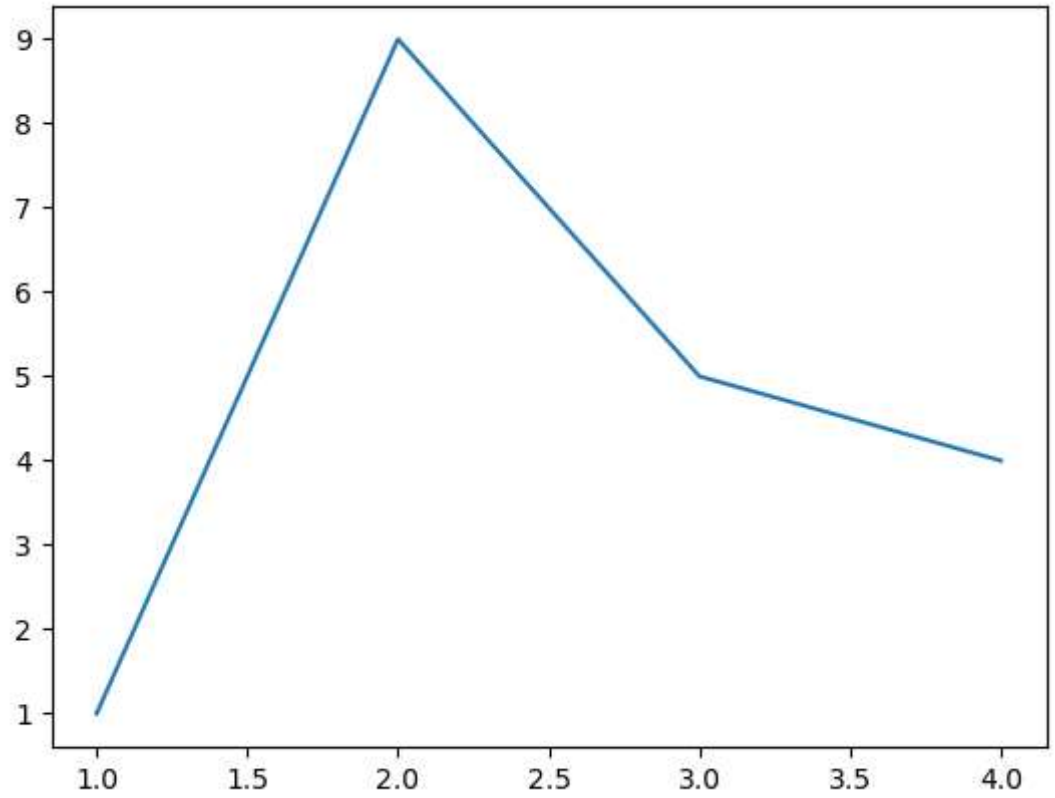
Matplotlib is open source and we can use it freely.

Most of the Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias:

In [1]:
```
import matplotlib.pyplot as plt
```

In [100…
```
import matplotlib.pyplot as plt
```

In [104…
```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y= [1,9,5,4] #(x,y)

plt.plot(x,y)
plt.show() #display
```



The x-axis is the horizontal axis.

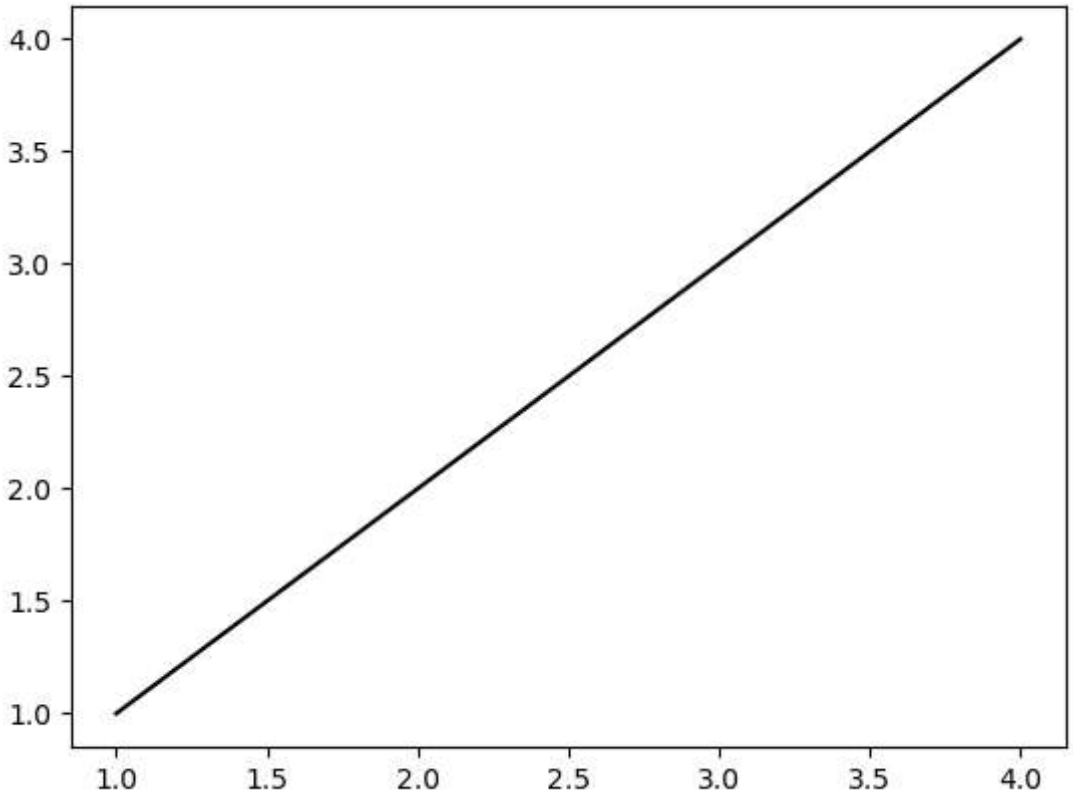
The y-axis is the vertical axis.

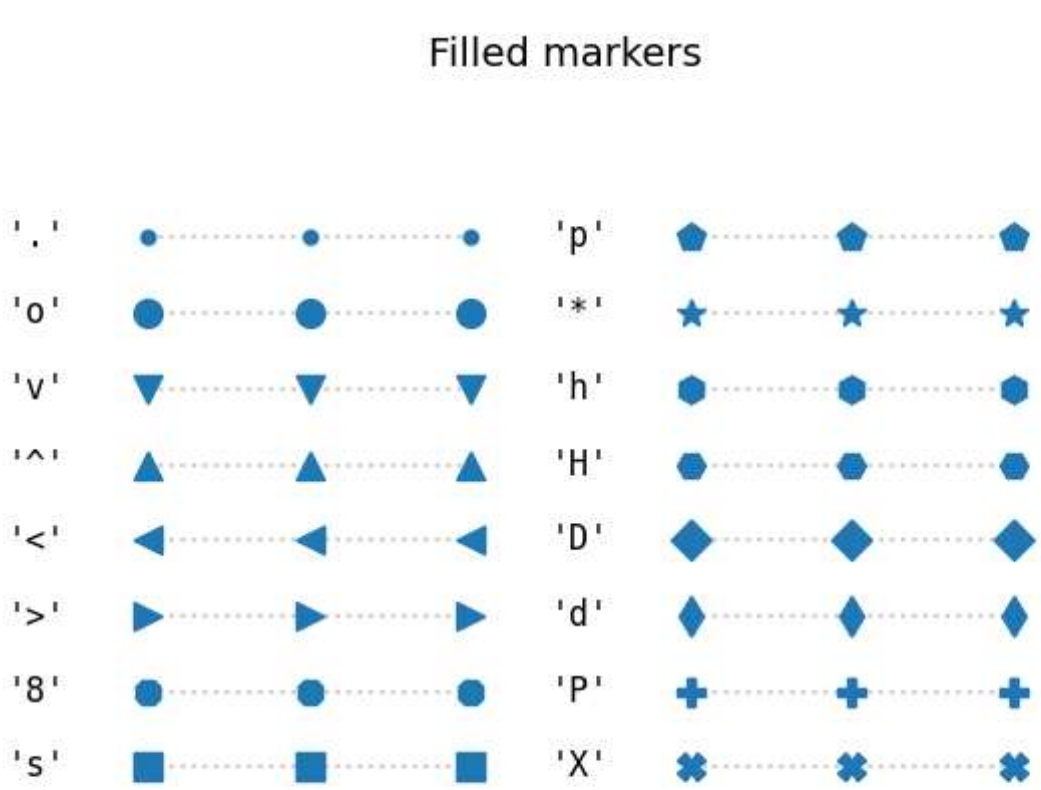
plot(x, y, color='green', marker='o',linestyle='dashed',linewidth=2,markersize=12)

Color Syntax	Description
'r'	Red
'g'	Green
'b'	Blue
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

In [108…

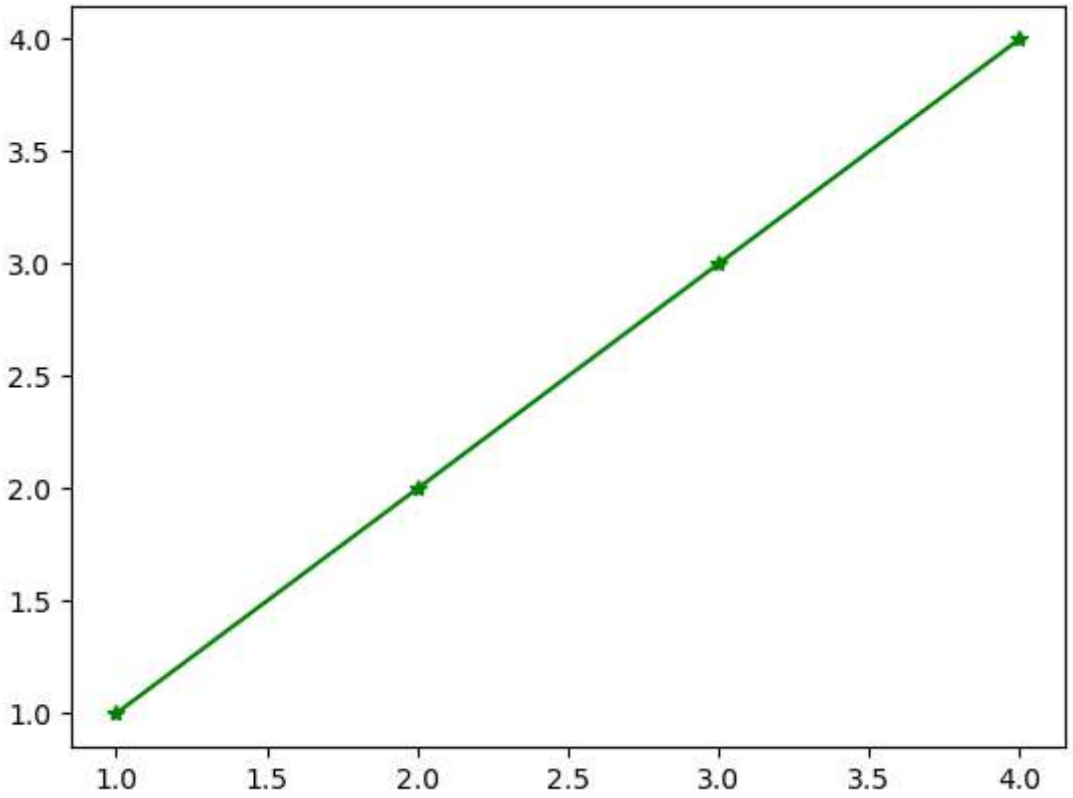
```
import matplotlib.pyplot as plt
import numpy as np

x = [1,2,3,4]
y= [1,2,3,4]

plt.plot(x, y,color="k",)
plt.show()
```


4.0
3.5
3.0
2.5
2.0
1.5
1.0
1.0
1.5
2.0
2.5
3.0
3.5
4.0



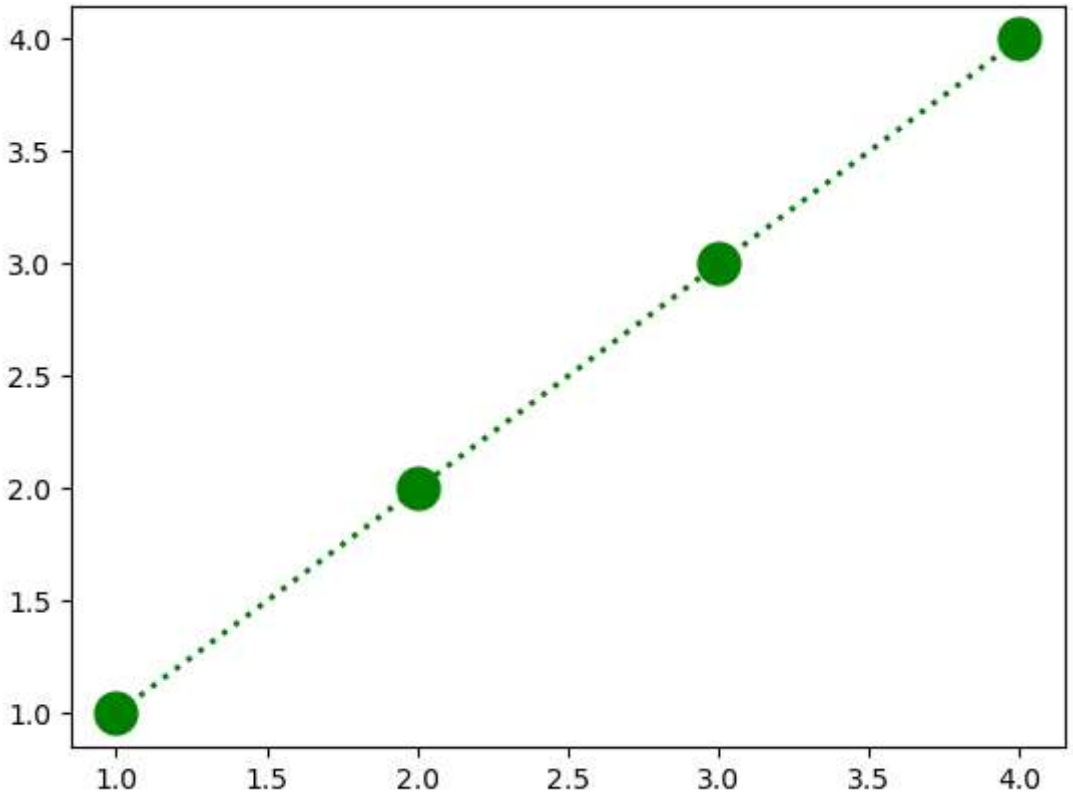
In [110…

```
import matplotlib.pyplot as plt
import numpy as np

x = [1,2,3,4]
y= [1,2,3,4]

plt.plot(x, y,color="g",marker="*",)
plt.show()
```



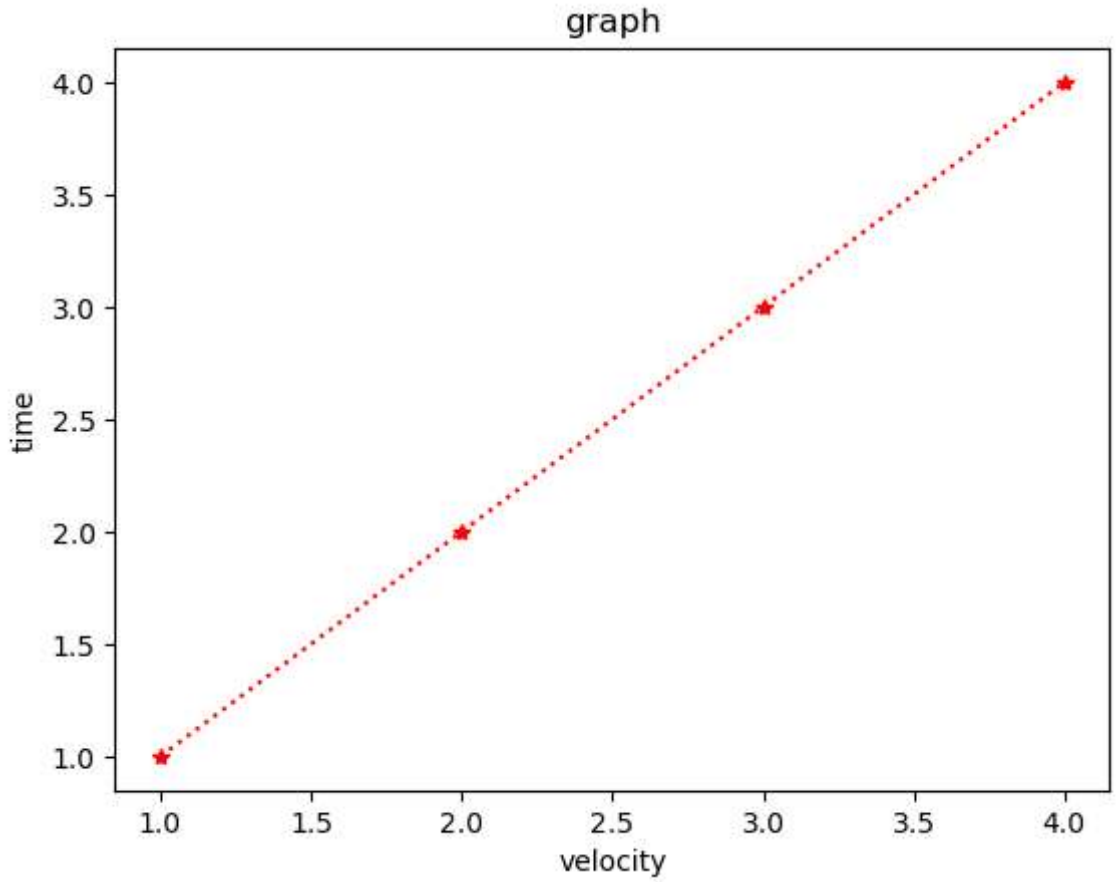
In [121…

```
import matplotlib.pyplot as plt
import numpy as np

x = [1,2,3,4]
y= [1,2,3,4]

plt.plot(x, y, color='green', marker='o',linestyle=":",linewidth=2,markersize=15)
plt.show()
```



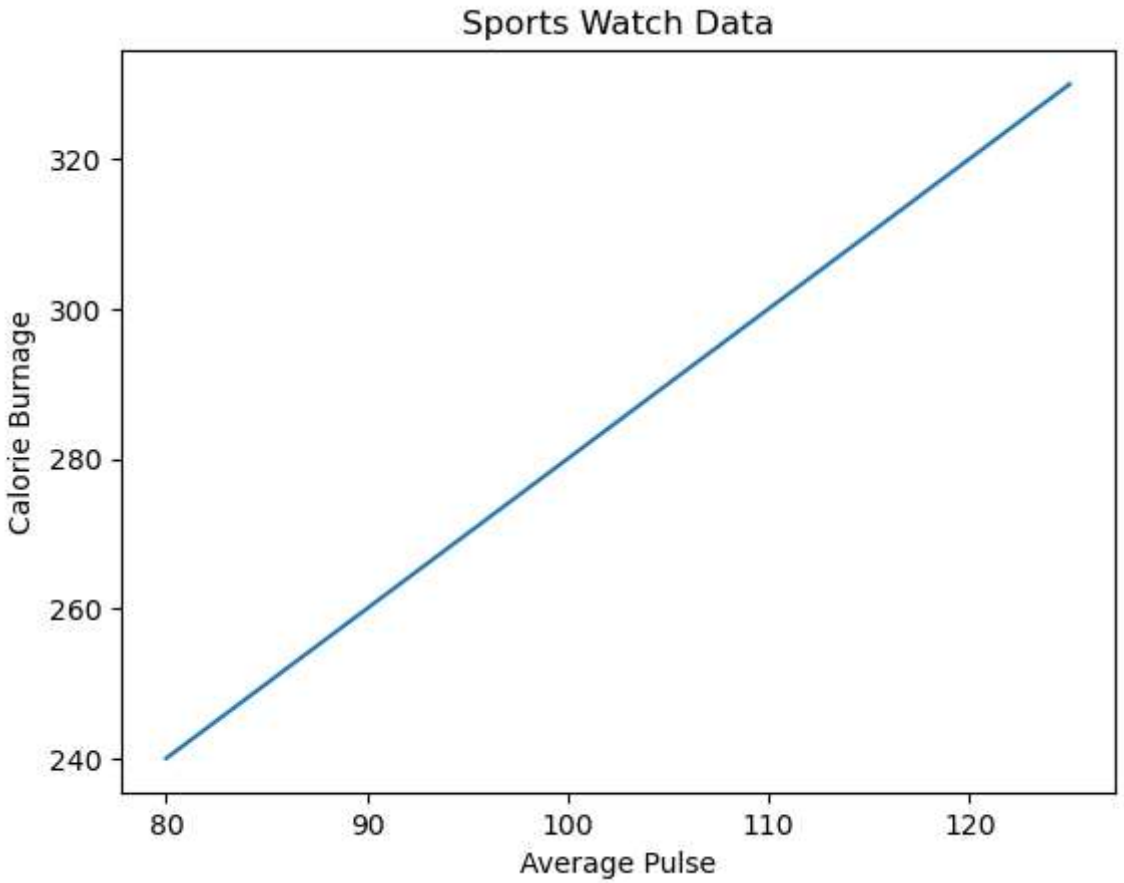
In [127…

```
import matplotlib.pyplot as plt
import numpy as np

x = [1,2,3,4]
y= [1,2,3,4]

plt.plot(x,y,"r*:")

plt.title("graph")
plt.xlabel("velocity")
plt.ylabel("time")
plt.show()
```



In [19]:

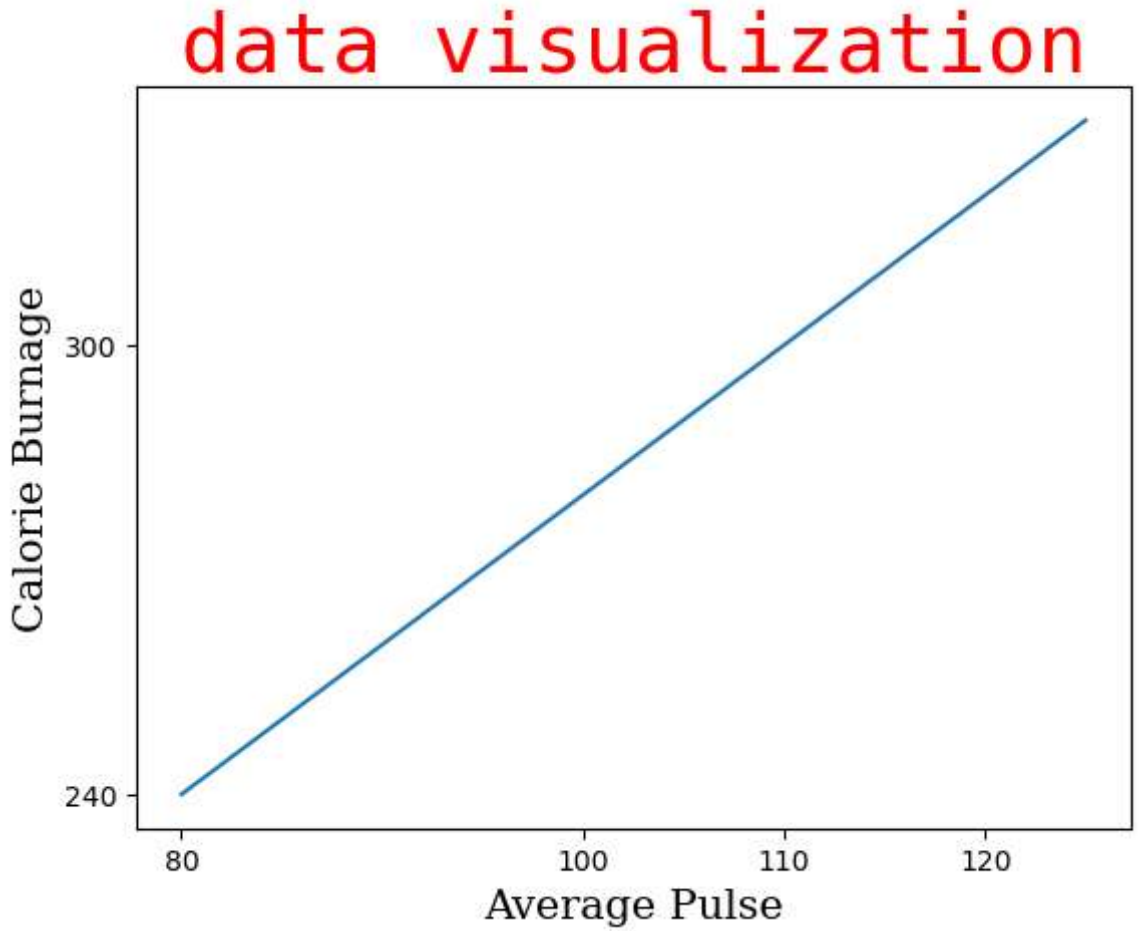
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)
plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")
plt.show()
```



In [143…

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

font1 = {'family':'monospace','color':'red','size':30}
font2 = {'family':'serif','color':'k','size':15}

plt.title("data visualization", fontdict = font1)
plt.xlabel("Average Pulse", fontdict = font2)
plt.ylabel("Calorie Burnage", fontdict = font2)
plt.xticks([80,100,110,120,150])
plt.yticks([240,300])
```

```
plt.plot(x, y)
plt.show()
```



some font names

'serif' 'sans-serif' 'monospace' 'cursive' 'fantasy' 'Arial' 'Times New Roman' 'Courier New' 'Verdana' 'Tahoma'

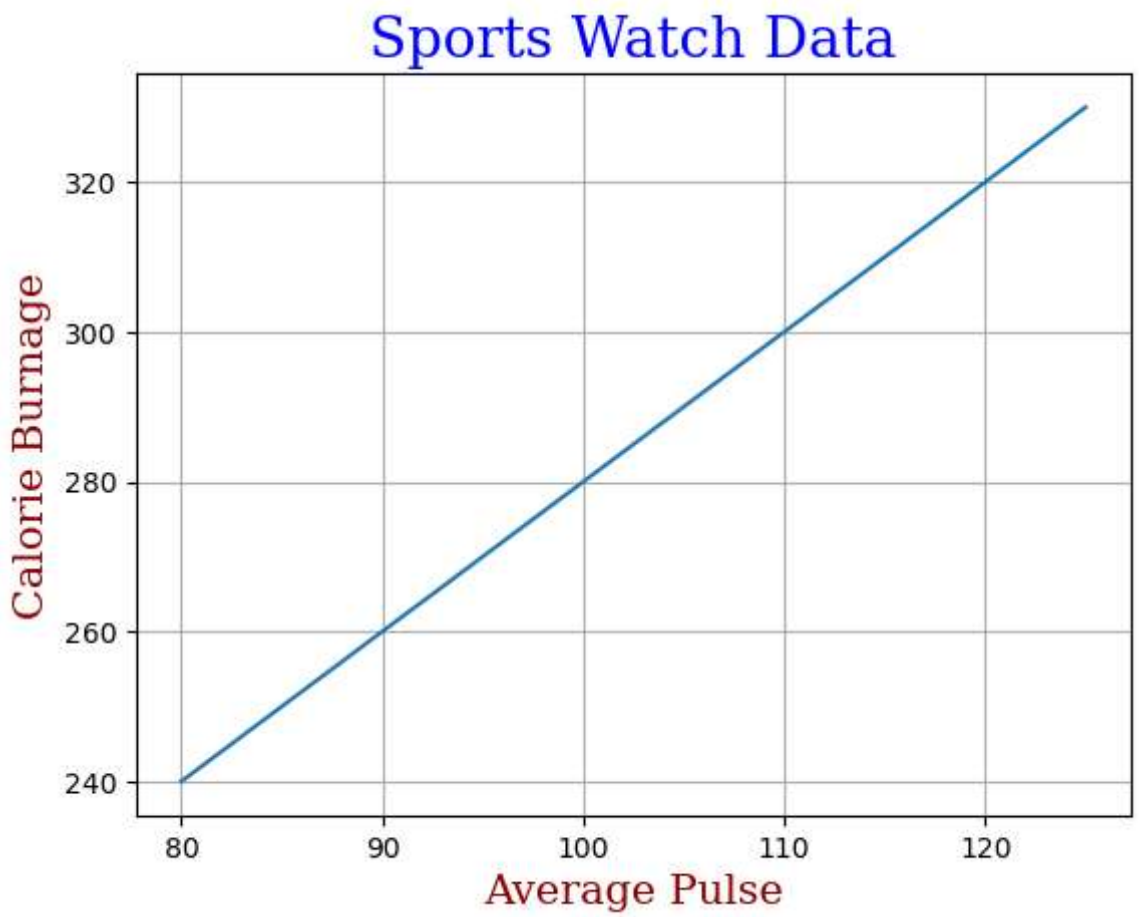
In [94]:

Add Grid Lines to a Plot

With Pyplot, you can use the grid() function to add grid lines to the plot.

In [133…

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

font1 = {'family':'serif','color':'blue','size':20}
font2 = {'family':'serif','color':'darkred','size':15}

plt.title("Sports Watch Data", fontdict = font1)
plt.xlabel("Average Pulse", fontdict = font2)
plt.ylabel("Calorie Burnage", fontdict = font2)

plt.grid()
#plt.grid(axis = 'x')
#plt.grid(axis = 'y')


plt.plot(x, y)
plt.show()
```

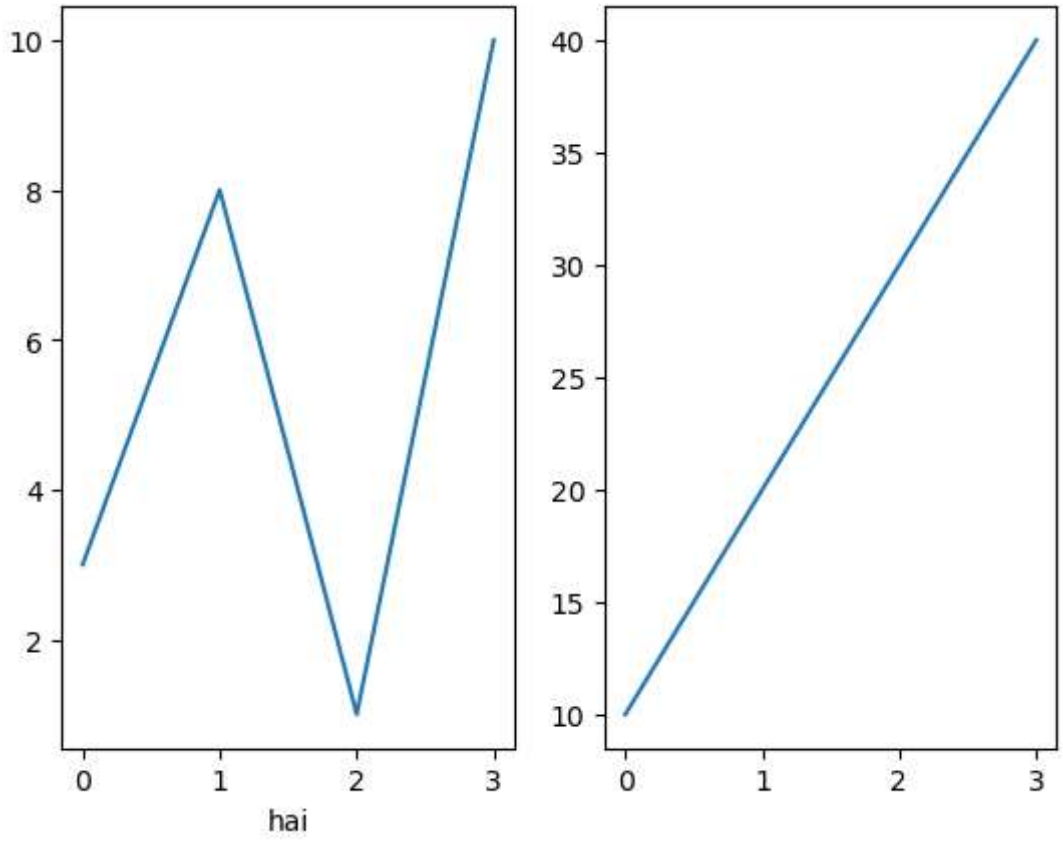


subplotes

In [137…

```
import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = [0, 1, 2, 3]
y = [3, 8, 1, 10]

plt.subplot(1, 2, 1)
plt.xlabel("hai")
plt.plot(x,y)

#plot 2:
```

```
x = [0, 1, 2, 3]
y = [10, 20, 30, 40]

plt.subplot(1, 2, 2)
plt.plot(x,y)

plt.show()
```



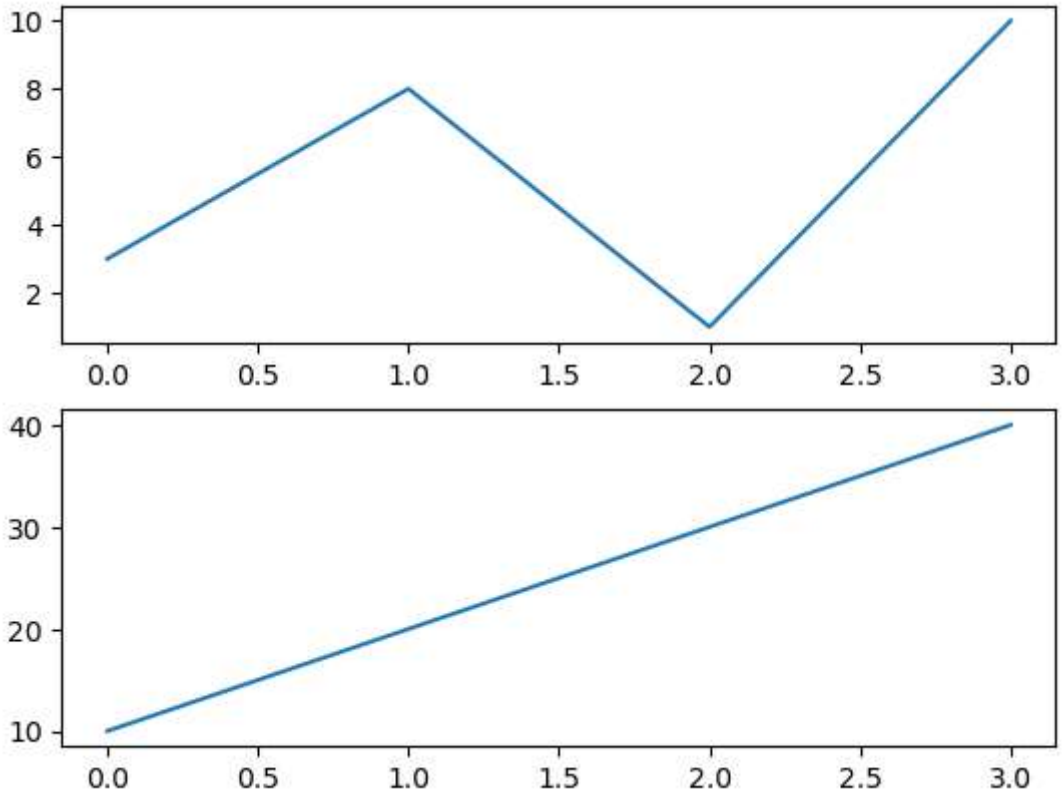
plt.subplot(1, 2, 1) #(rows,columns,which graph)

the figure has 1 row, 2 columns, and this plot is the first plot.

plt.subplot(1, 2, 2)

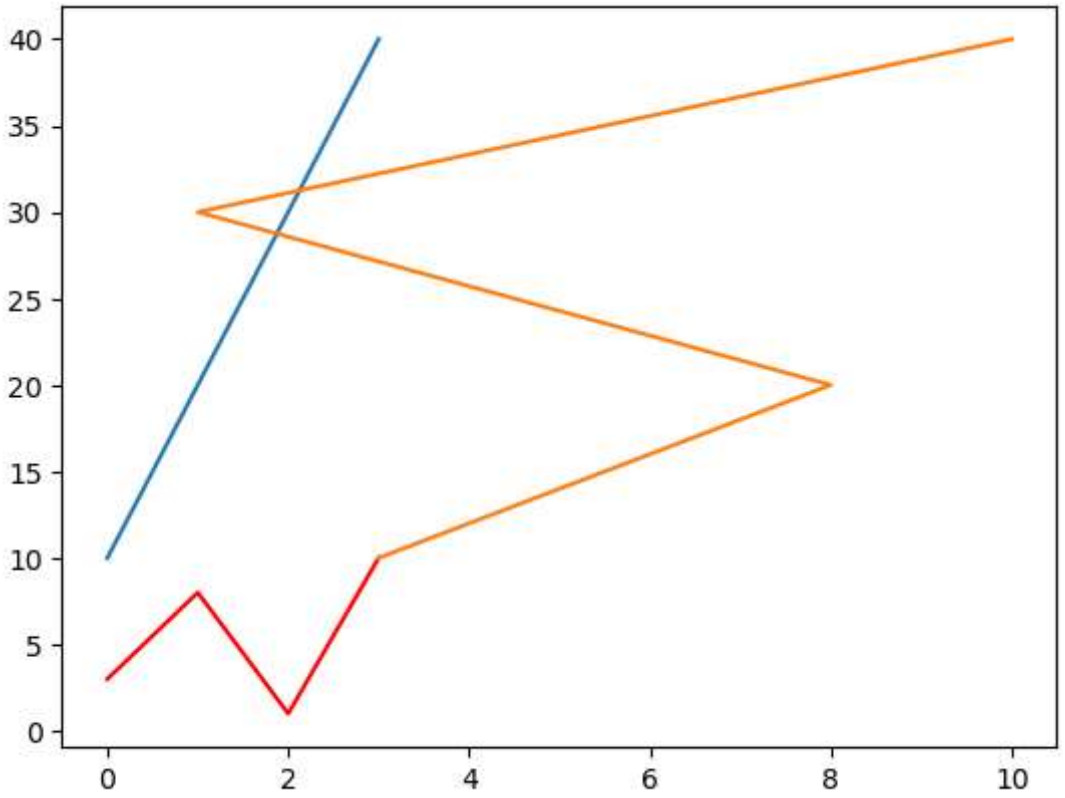
the figure has 1 row, 2 columns, and this plot is the second plot.

In [26]:
```
import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 1, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 1, 2)
plt.plot(x,y)

plt.show()import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 1, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 1, 2)
plt.plot(x,y)

plt.show()
```



In [151…

```
import matplotlib.pyplot as plt
import numpy as np

x1 = np.array([0, 1, 2, 3])
y1 = np.array([3, 8, 1, 10])


x2 = np.array([0, 1, 2, 3])
y2 = np.array([10, 20, 30, 40])

plt.plot(x1,y1,color="red")
plt.plot(x2,y2)
plt.plot(y1,y2)

plt.show()
```



In [ ]:
```
plt.Subplot(2,1,1)
plt.Subplot(2,1,2)
```

Creating Scatter Plots

A scatter plot is a type of data visualization that is used to display the relationship between two continuous variables.

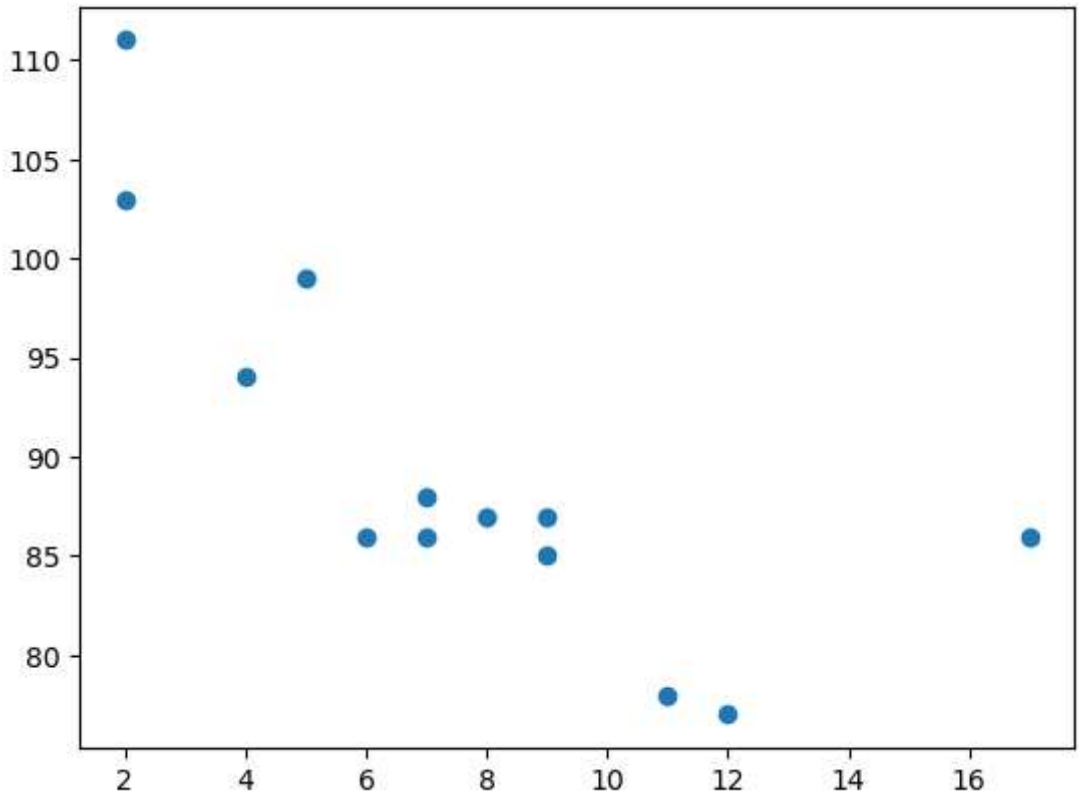
It is particularly useful for identifying patterns, trends, and relationships between variables.

use the scatter() function to draw a scatter plot.

The scatter() function plots one dot for each observation. It needs two arrays of the same length, one for the values of the x-axis, and one for values on the y-axis:

In [152…

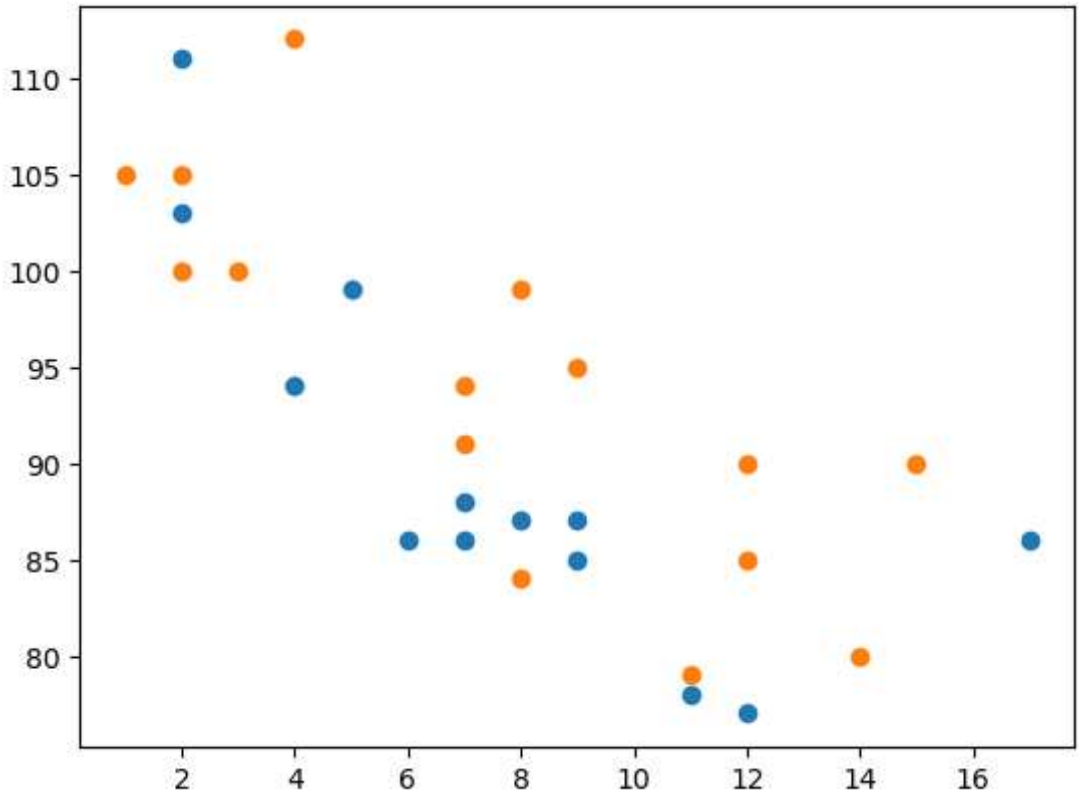
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x,y)
plt.show()
```



In [28]:

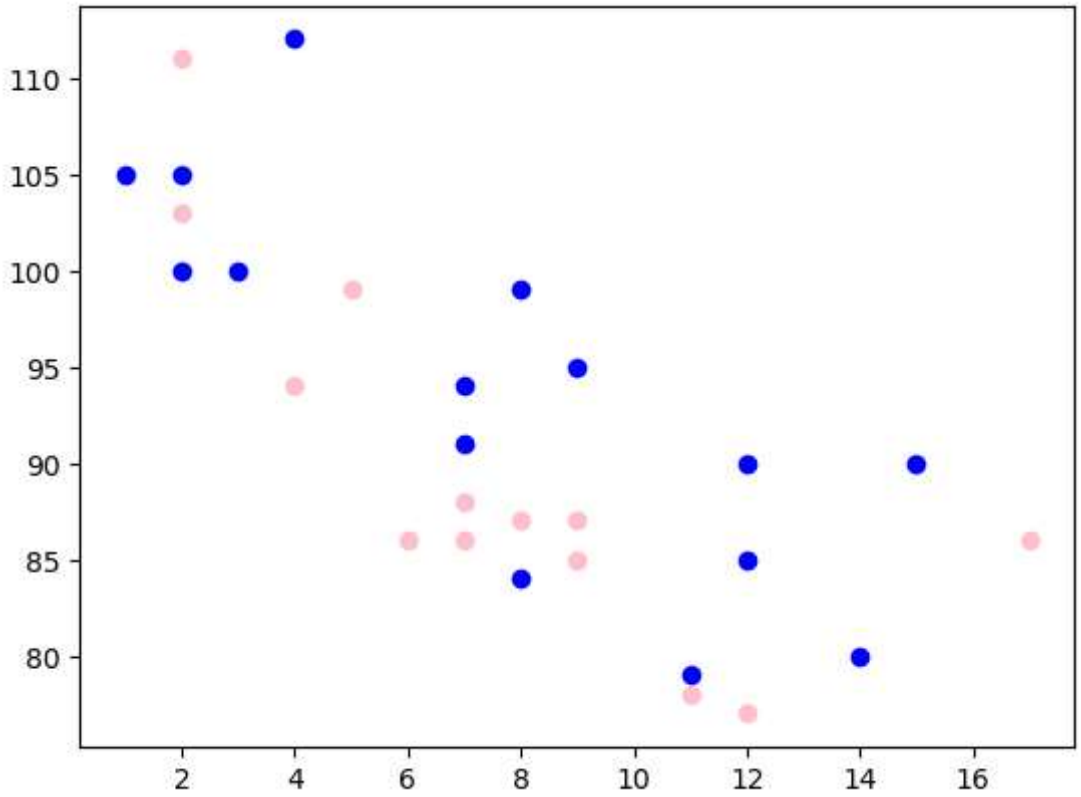
```
import matplotlib.pyplot as plt
import numpy as np

#day one, the age and speed of 13 cars:
x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
plt.scatter(x, y)

#day two, the age and speed of 15 cars:
```

```
x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
plt.scatter(x, y)

plt.show()
```



In [30]:
```
#we can set colors

import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
plt.scatter(x, y, color = 'pink')

x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
plt.scatter(x, y, color = 'blue')
```

```
plt.show()
```



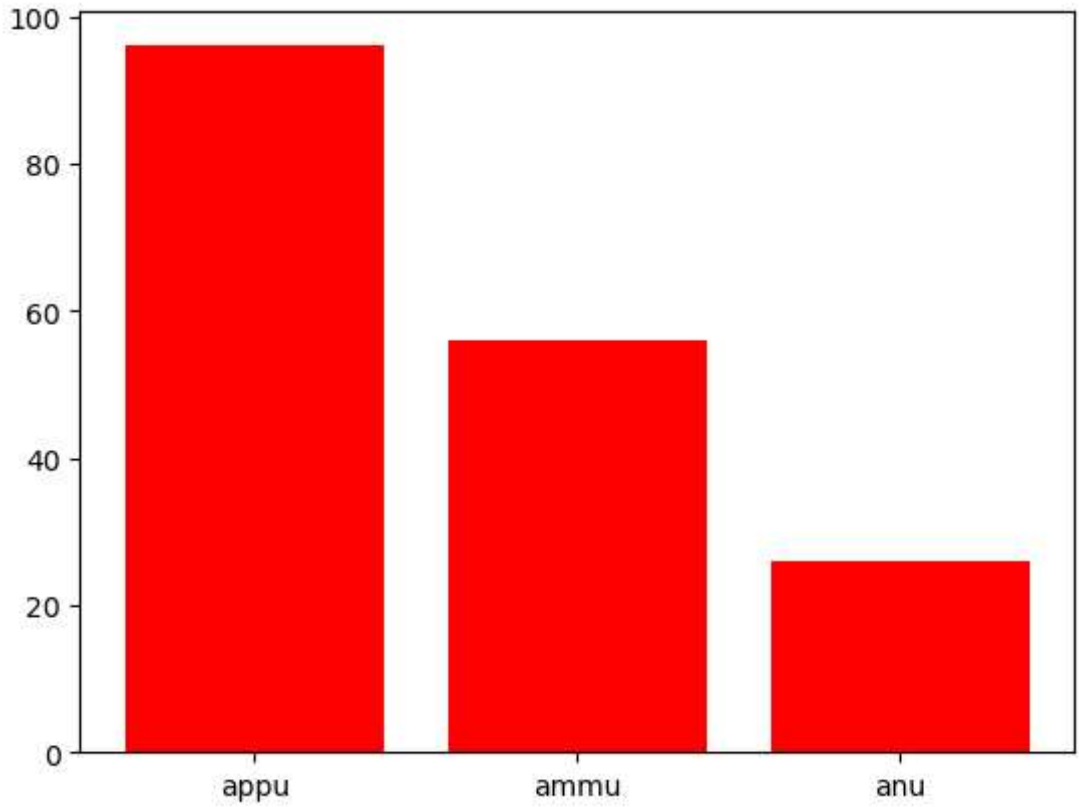
bar plot

A bar plot, also known as a bar chart, is a type of data visualization that represents categorical data with rectangular bars.

Categorical Representation: Bar plots are used to represent categorical data, where each category is represented by a bar. Frequency or Value Display: The length or height of each bar represents the frequency or value of the category it represents. Comparative Analysis: Bar plots are useful for comparing the values of different categories. They make it easy to visually identify the largest or smallest values and compare the relative sizes of categories. Grouped or Stacked: Bar plots can be grouped or stacked to represent multiple variables or subcategories within each main category. Horizontal or Vertical Orientation: Bar plots can be oriented either horizontally or vertically, depending on the data and the preferred visualization style.

In [153…

```
import matplotlib.pyplot as plt
import numpy as np

name = ["appu","ammu","anu"]
marks = [96,56,26]

plt.bar(name, marks, color = "red")
plt.show()
```

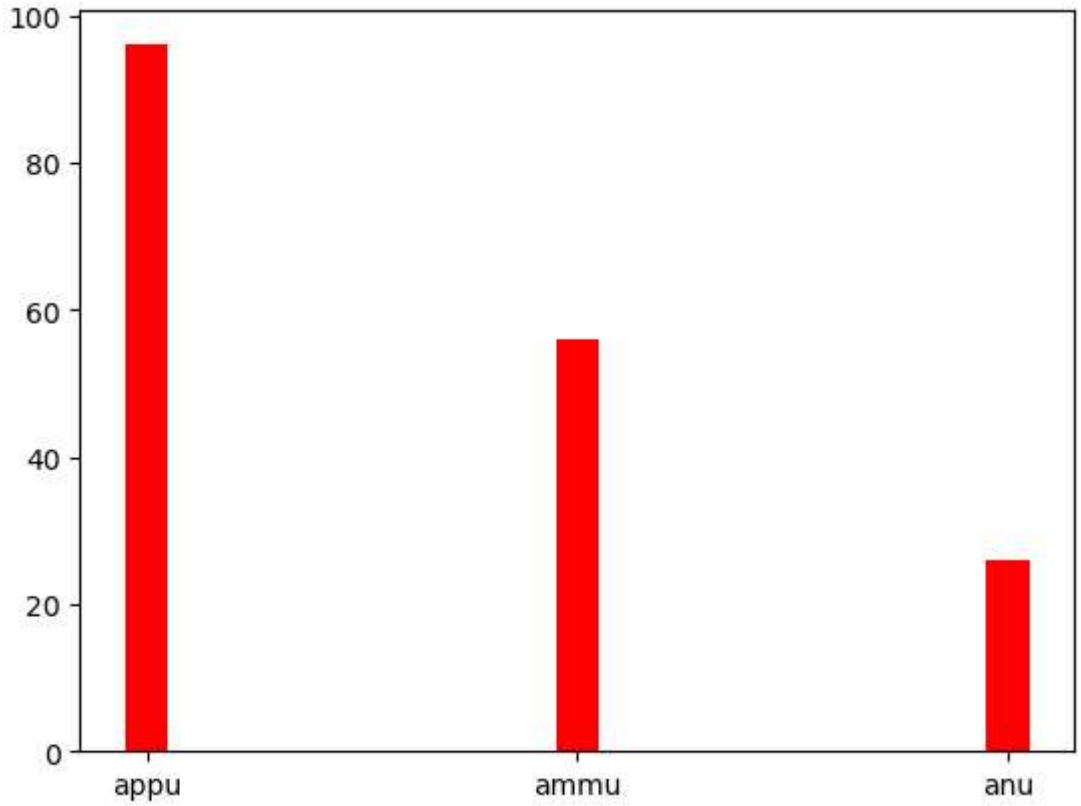


In [154…

```
plt.bar(name, marks, width = 0.1,color="red")
```

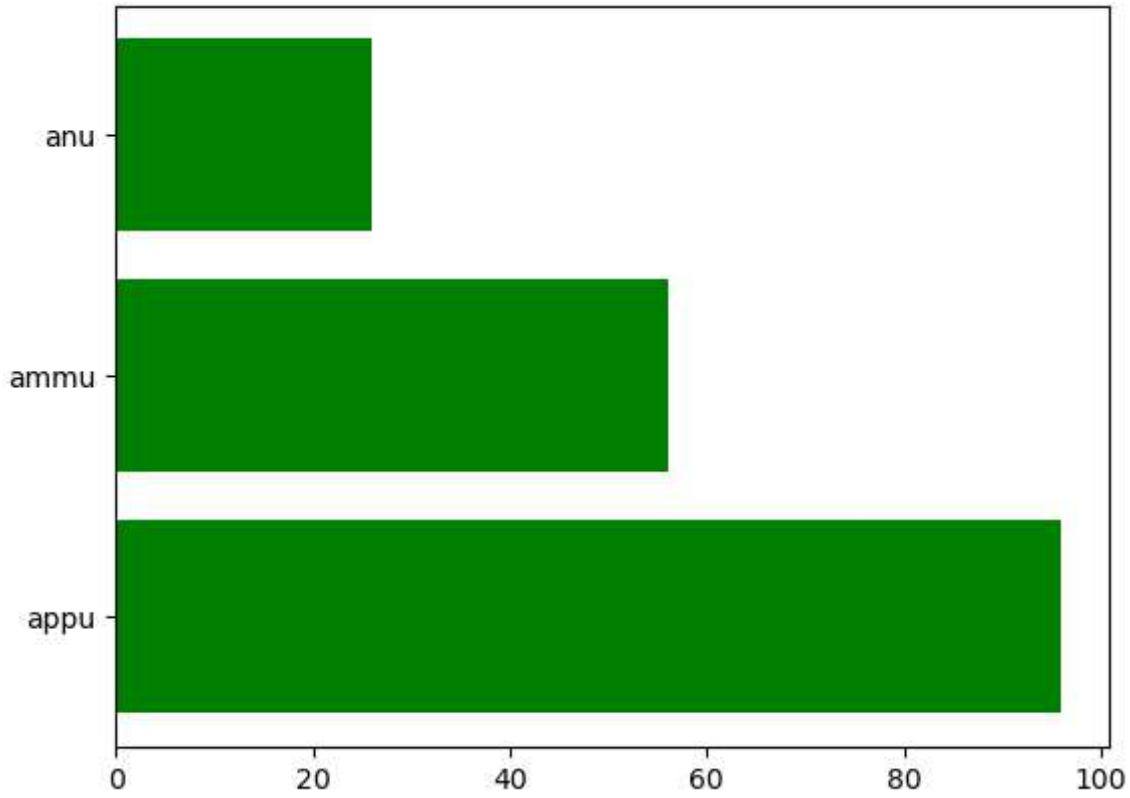
Out[154]:

```
<BarContainer object of 3 artists>
```



In [155…

```
plt.barh(name, marks,color="green")
plt.show()
```

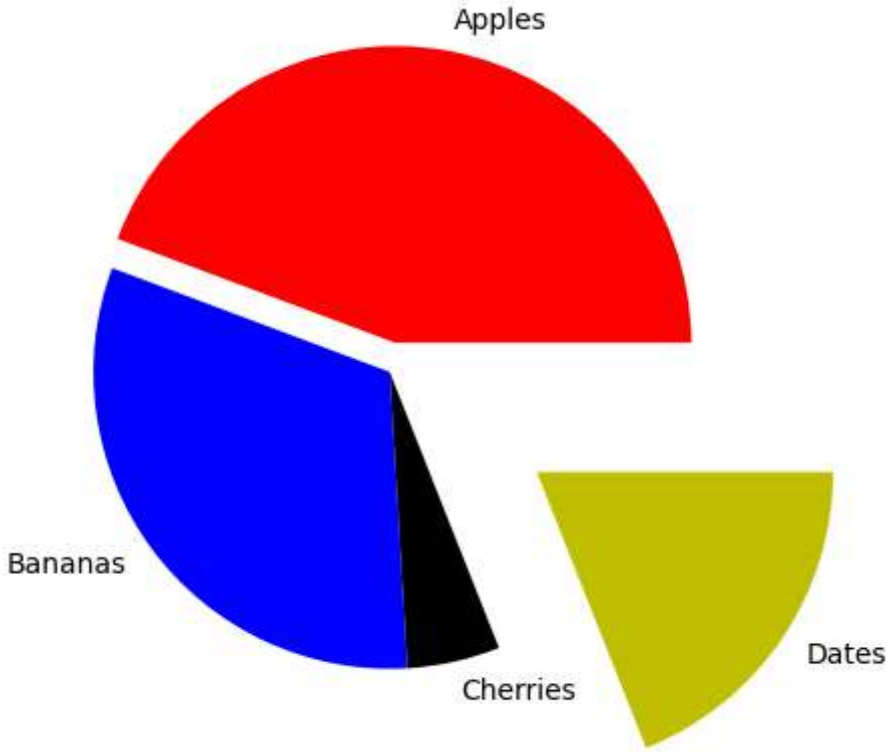


A pie chart

is a circular statistical graphic that is divided into slices to illustrate numerical proportions. Each slice represents a proportionate part of the whole, and the size of each slice is proportional to the quantity it represents. Pie charts are useful for showing the composition of a whole and for comparing the parts to the whole or to each other.

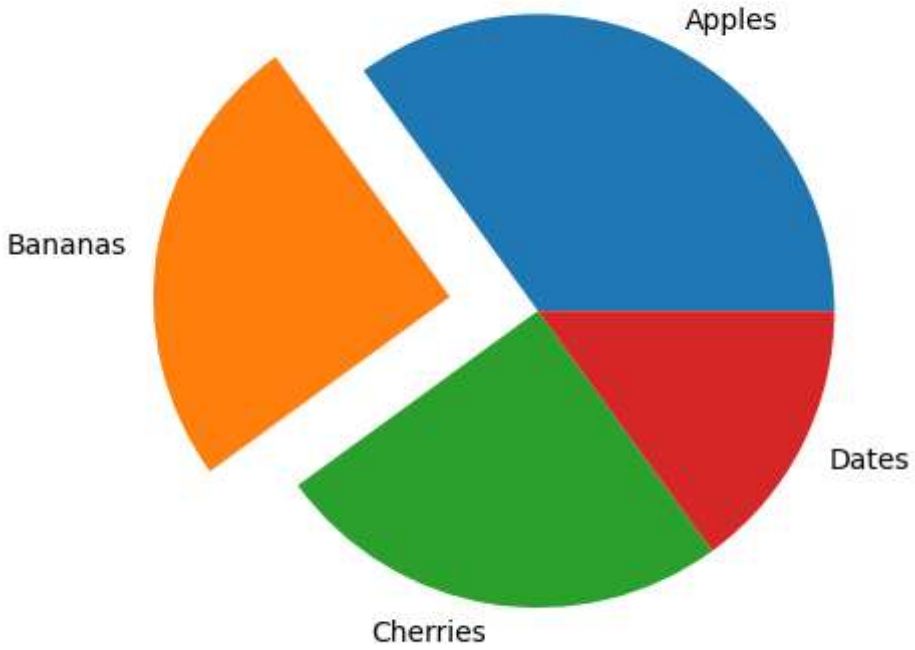
In [168…

```
import matplotlib.pyplot as plt

val = [35, 25, 4, 15]
x= ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(val,labels=x,colors=["r","b","k","y"],explode=[0.1,0,0,.6])
plt.show()
```



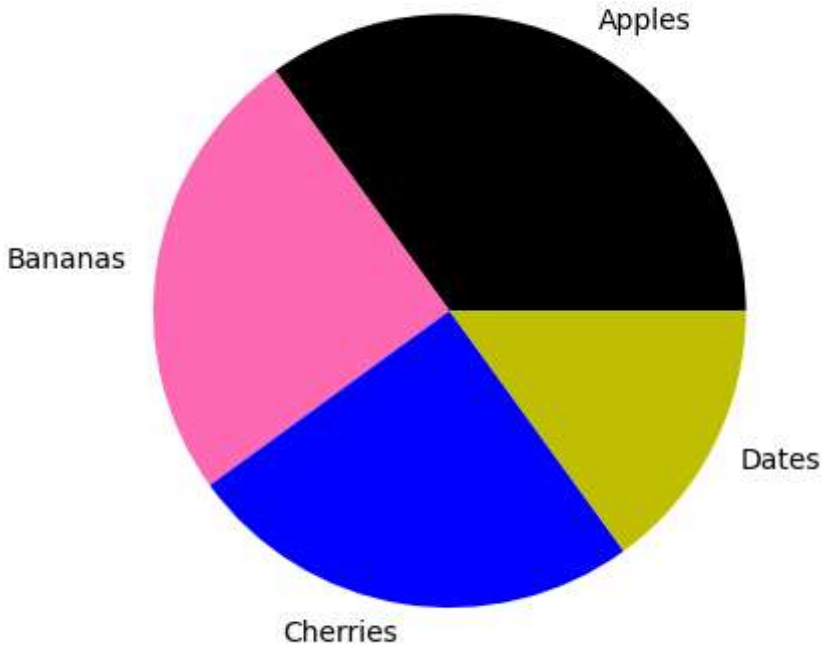
In [169…

```
y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0, 0.3, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode)
plt.show()
```



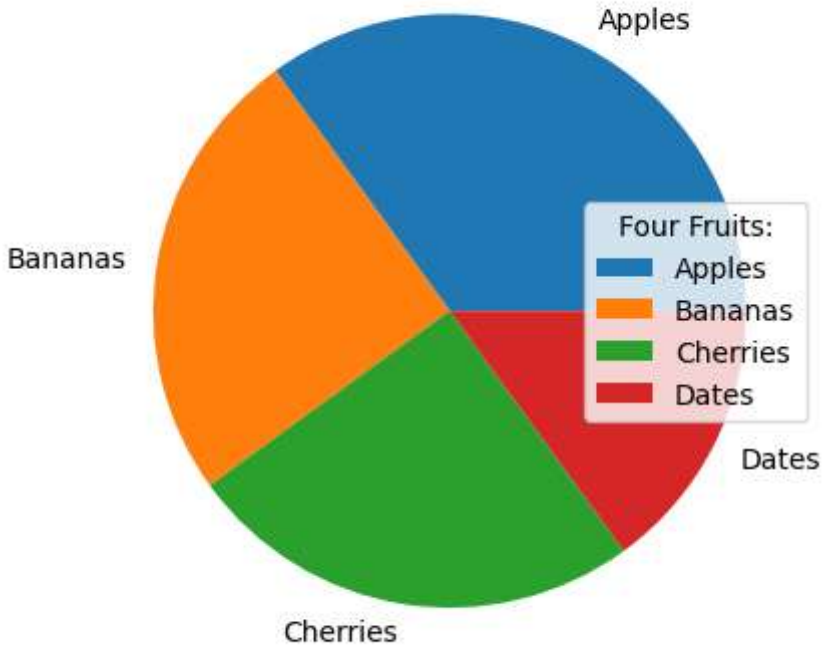
In [170…

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
mycolors = ["black", "hotpink", "b", "y"]

plt.pie(y, labels = mylabels, colors = mycolors)
plt.show()
```



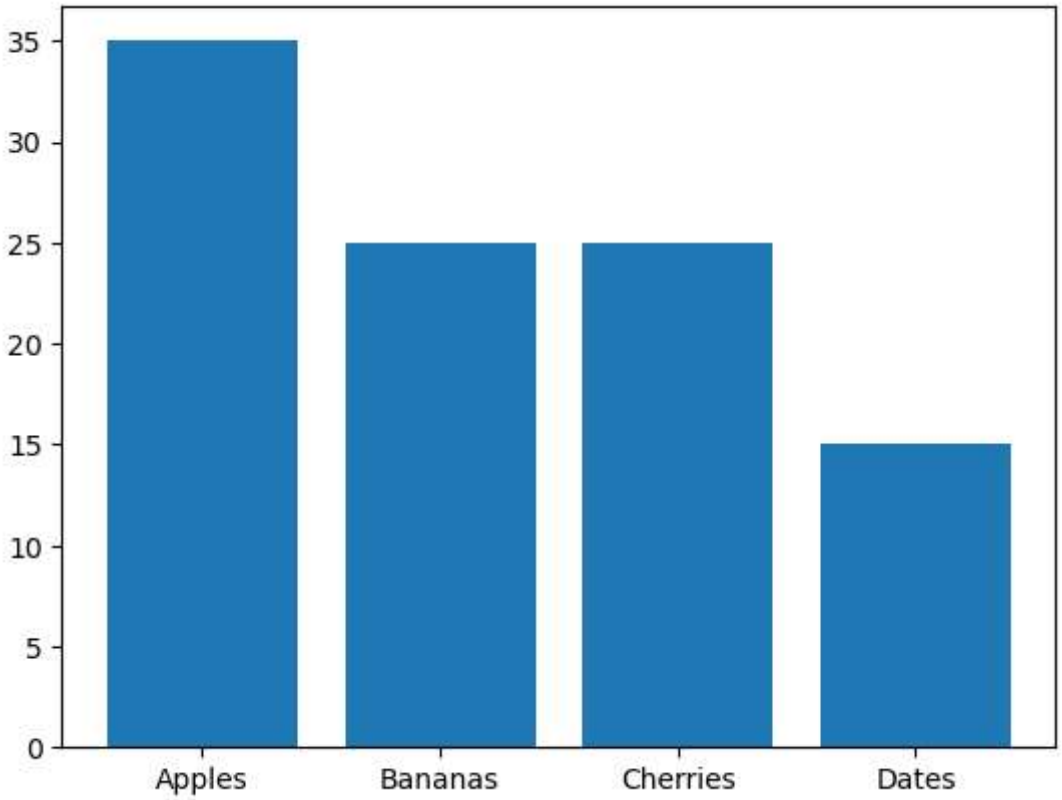
In [175…

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.legend(title = "Four Fruits:",loc="right")
plt.show()
```



In [177…
```
y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
plt.bar(mylabels,y)
```

Out[177]:
```
<BarContainer object of 4 artists>
```



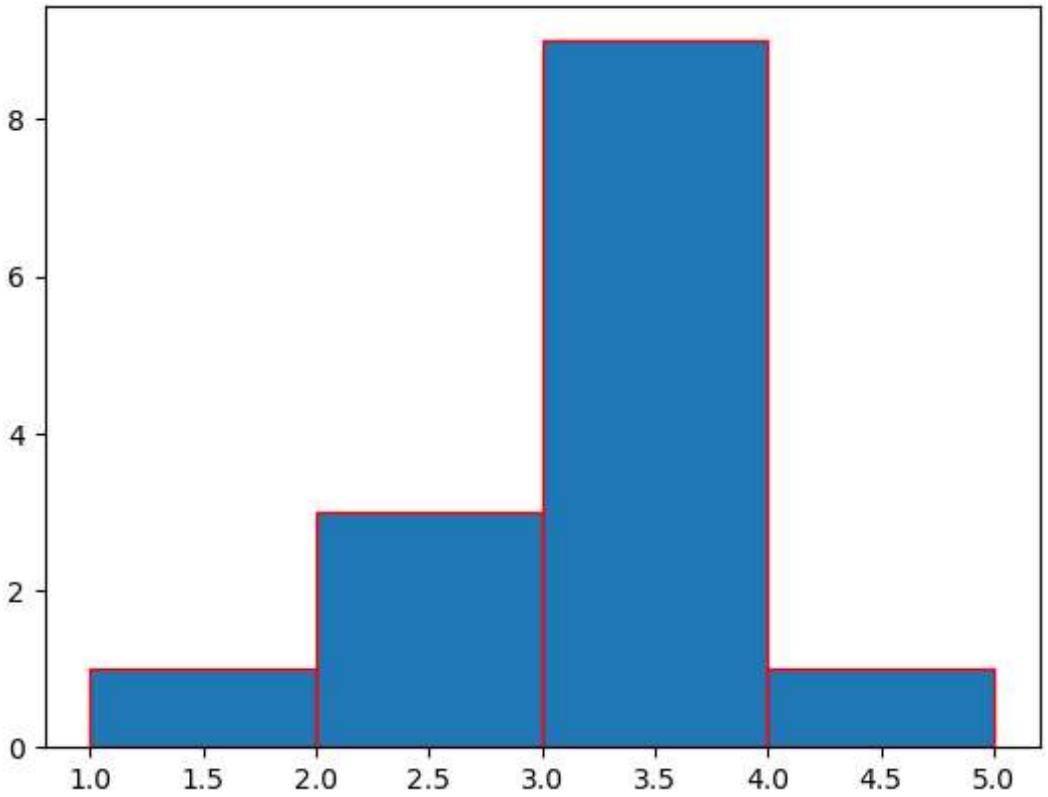
Histogram

A histogram is a graph showing frequency distributions.

It is a graph showing the number of observations within each given interval.

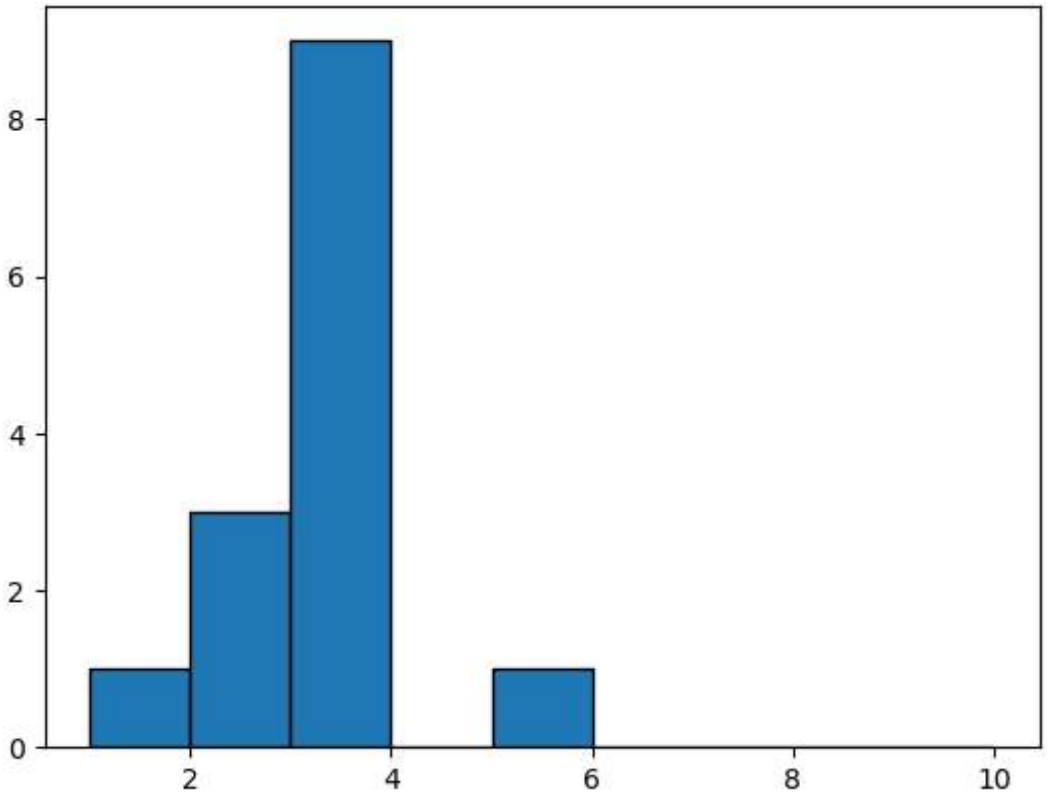
In [189…

```
marks=[.5,1,5,3.5,3.3,3,3,3,3,3,2.5,2,3.5,3.5,3.5,2]
plt.hist(marks,edgecolor="red", bins=[1,2,3,4,5])
plt.show()
```

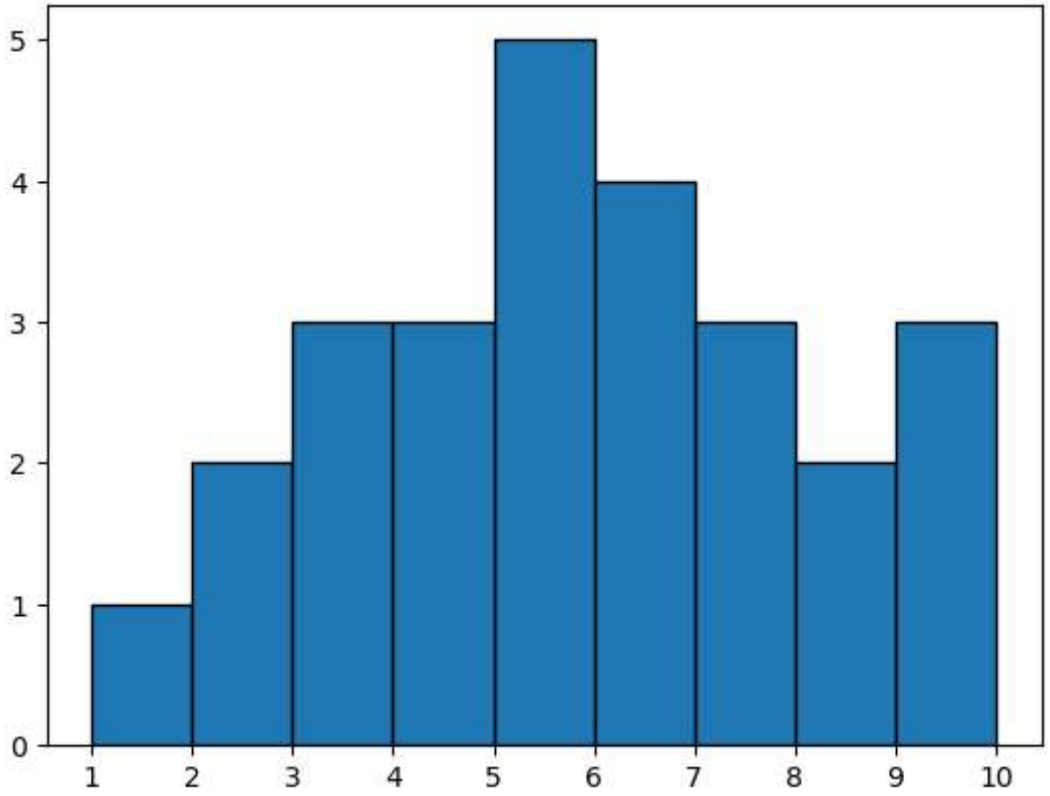


In [190…

```
plt.hist(marks, bins=range(1, 11), edgecolor="black")
plt.show()
```



In [58]:
```
plt.hist(marks, bins=range(1, 11), edgecolor="black")
plt.xticks(range(1, 11))  # Set the ticks from 1 to 10
plt.show()
```



A box plot,

also known as a box-and-whisker plot, is a graphical representation of the distribution of a dataset. It displays key statistical information including the minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum values. Additionally, it can show outliers - data points that fall significantly outside the main distribution.

Box plots are commonly used for:

Visualizing Data Distribution:

Box plots provide a quick visual summary of the central tendency, spread, and skewness of a dataset.

Identifying Outliers:

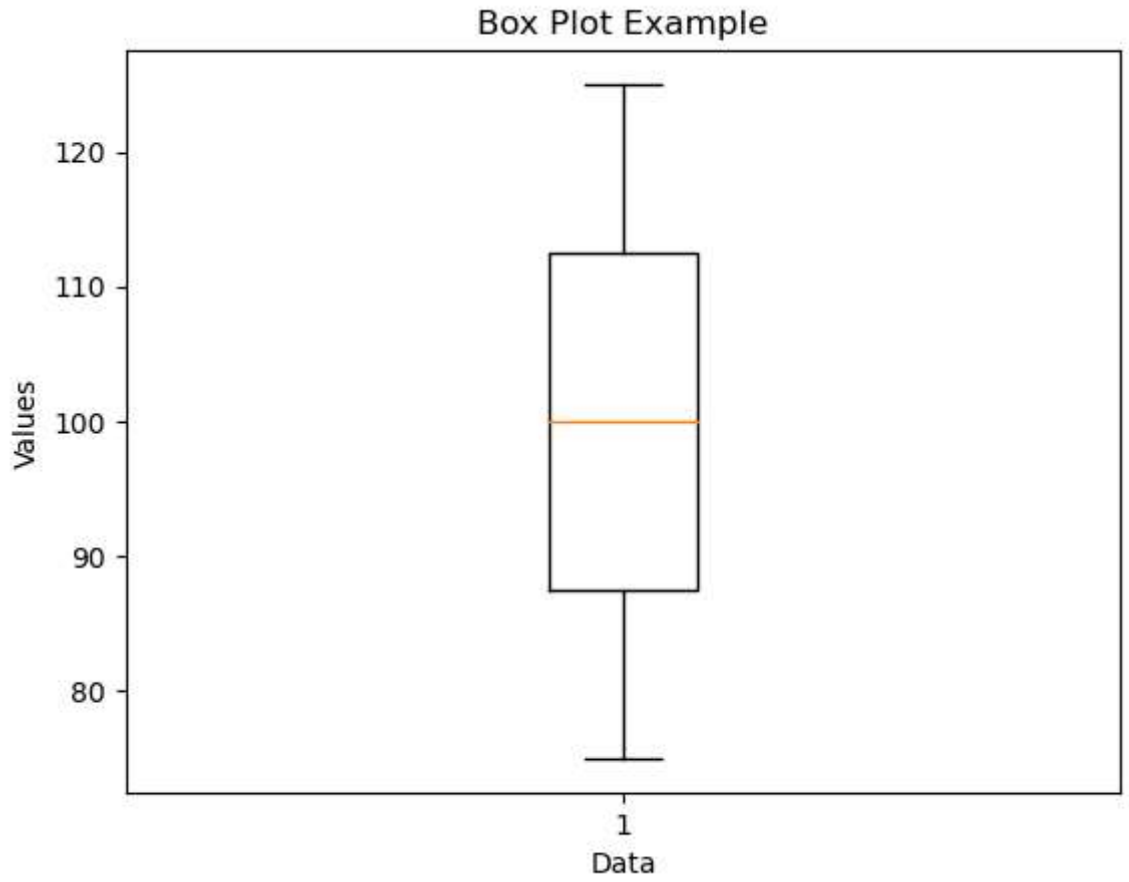
Outliers, which are data points that lie significantly outside the bulk of the data, are visually highlighted in box plots, making them easy to identify.

Detecting Skewness and Symmetry:

The shape of the box plot can provide insights into the symmetry or skewness of the data distribution. For example, if the median line is not centered within the box, it suggests skewness.

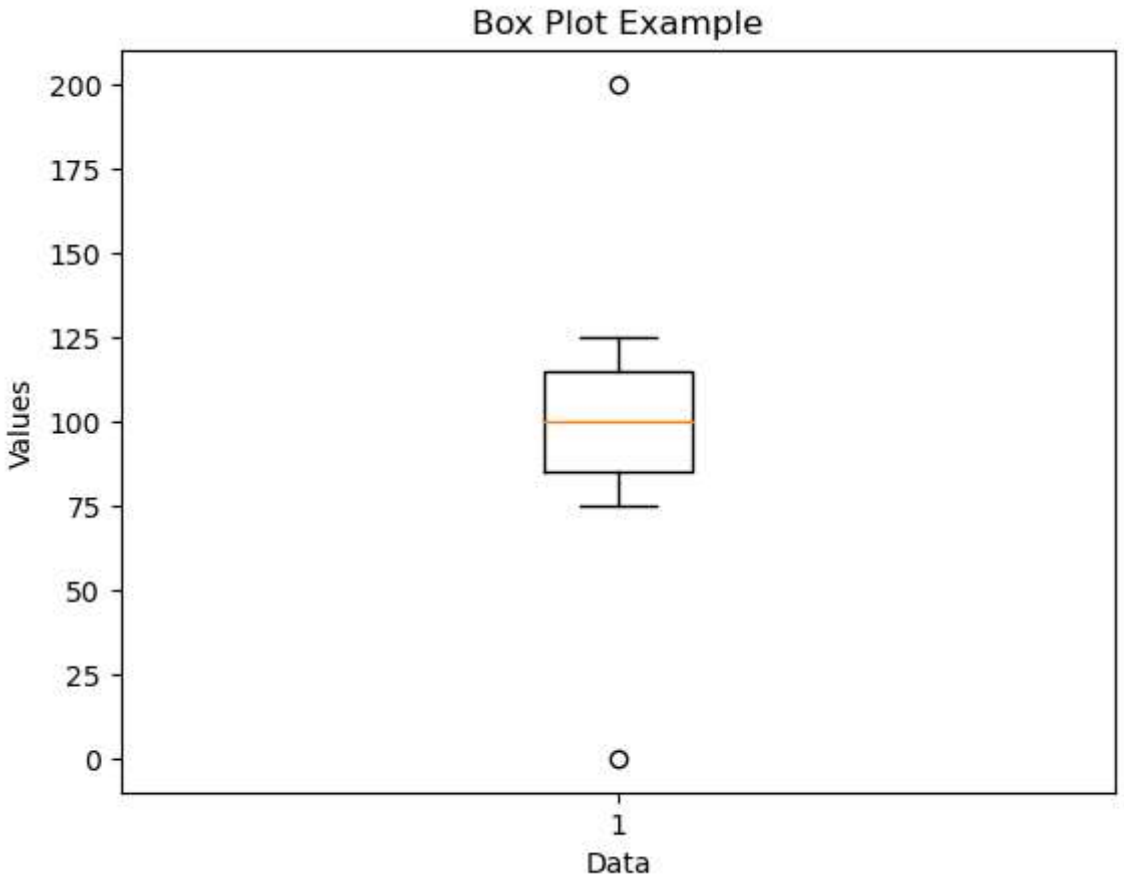
In [191…

```
import matplotlib.pyplot as plt

# Example data
data = [75, 80, 85, 90, 95, 100, 105, 110, 115, 120, 125]

# Create a box plot
plt.boxplot(data)

# Add labels and title
plt.xlabel('Data')
plt.ylabel('Values')
plt.title('Box Plot Example')

# Show the plot
plt.show()
```



In [192…

```python
import matplotlib.pyplot as plt

# Example data
data = [75, 80, 85, 90, 95, 100, 105, 110, 115, 120, 125,200,0]

# Create a box plot
plt.boxplot(data)

# Add labels and title
plt.xlabel('Data')
plt.ylabel('Values')
plt.title('Box Plot Example')

# Show the plot
plt.show()
```



In [65]:
```
import pandas as pd
```

In [193…
```
data=pd.read_csv("C:/Users/HP/Downloads/students_perfomance_in exam.csv - Sheet1.csv")
```

In [194…
```
data
```

Out[194]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75
...	...	...	...	...	...	...	...	...
120	female	group C	bachelor's degree	standard	completed	79	92	89
121	male	group B	associate's degree	standard	completed	91	89	92
122	female	group C	some college	standard	completed	88	93	93
123	male	group D	high school	free/reduced	none	63	57	56
124	male	group E	some college	standard	none	83	80	73

125 rows × 8 columns

In [195...
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 125 entries, 0 to 124
Data columns (total 8 columns):
 #   Column                       Non-Null Count  Dtype 
---  ------                       --------------  ----- 
 0   gender                       125 non-null    object
 1   race/ethnicity               125 non-null    object
 2   parental level of education  125 non-null    object
 3   lunch                        125 non-null    object
 4   test preparation course      125 non-null    object
 5   math score                   125 non-null    int64 
 6   reading score                125 non-null    int64 
 7   writing score                125 non-null    int64 
dtypes: int64(3), object(5)
memory usage: 7.9+ KB
```

In [196...
```
data.describe()
```

Out[196]:

	math score	reading score	writing score
count	125.000000	125.00000	125.000000
mean	63.008000	66.64000	65.464000
std	16.425539	16.28001	17.160318
min	0.000000	17.00000	10.000000
25%	53.000000	56.00000	54.000000
50%	63.000000	68.00000	67.000000
75%	74.000000	78.00000	76.000000
max	99.000000	100.00000	100.000000

In [197…
```
data.isnull().sum()
```

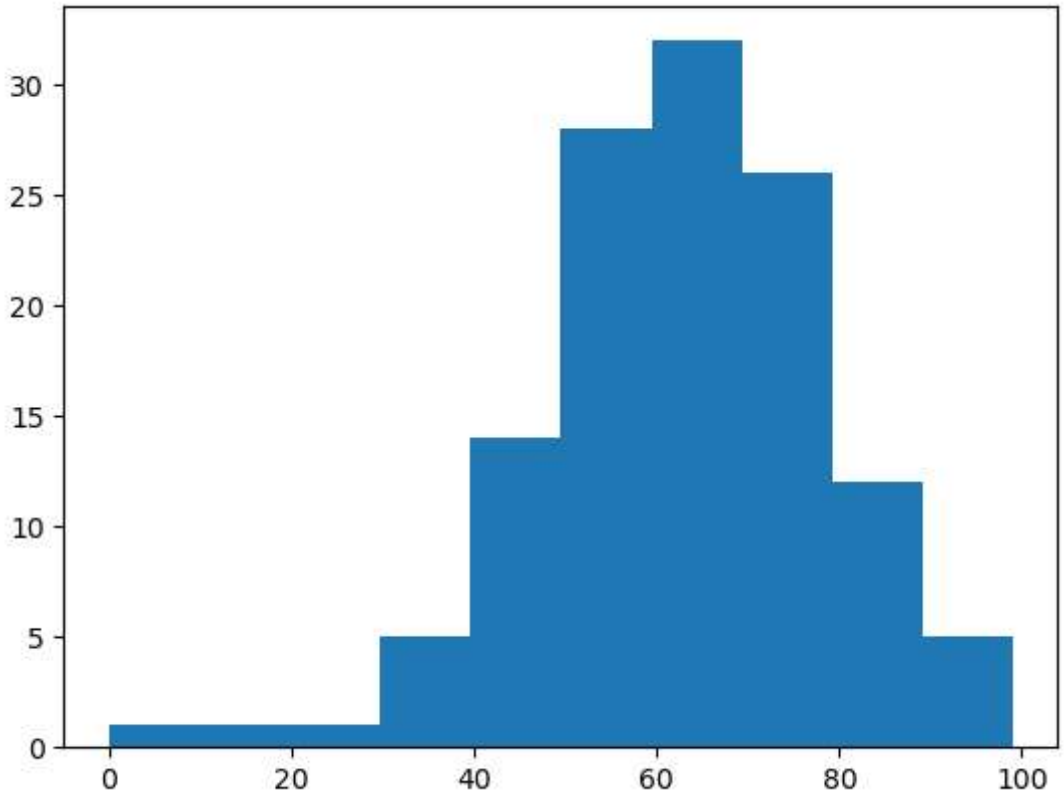
Out[197]:
```
gender                       0
race/ethnicity               0
parental level of education  0
lunch                        0
test preparation course      0
math score                   0
reading score                0
writing score                0
dtype: int64
```

In [198…
```
data.duplicated().sum()
```

Out[198]:
```
0
```

In [200…
```
y=data["math score"]
plt.hist(y)
```

Out[200]:
```
(array([ 1.,  1.,  1.,  5., 14., 28., 32., 26., 12.,  5.]),
 array([ 0. ,  9.9, 19.8, 29.7, 39.6, 49.5, 59.4, 69.3, 79.2, 89.1, 99. ]),
 <BarContainer object of 10 artists>)
```

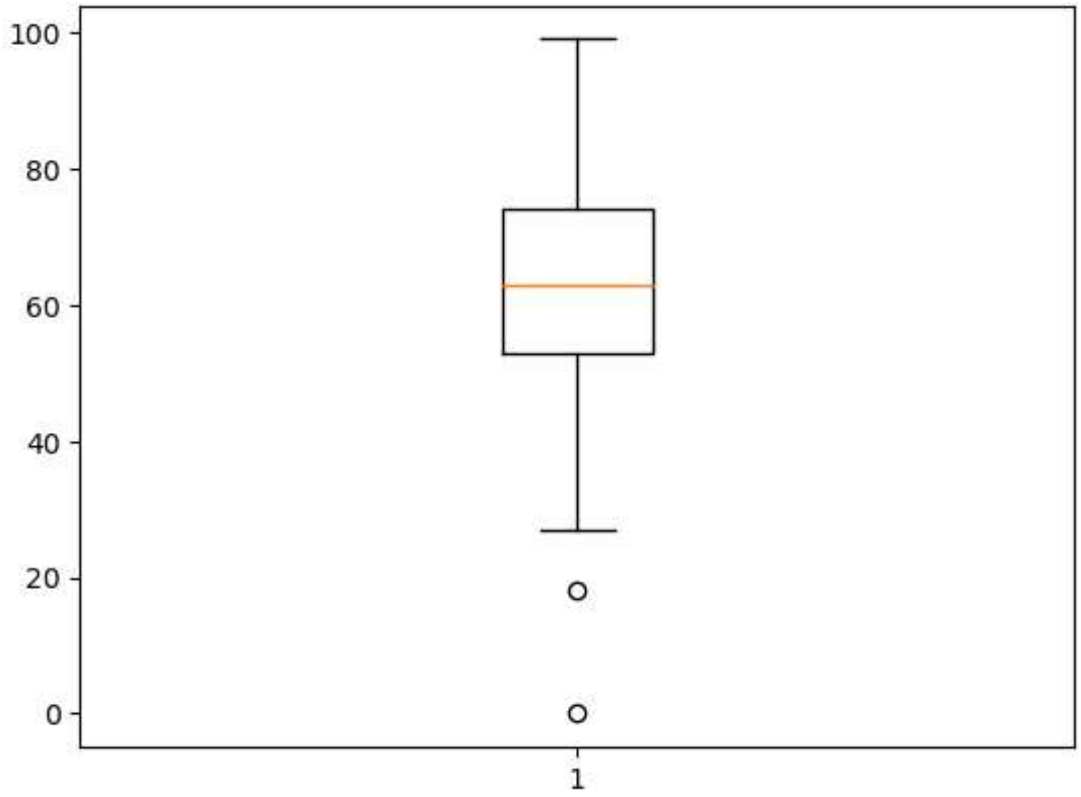


In [201…

```
plt.boxplot(y)
plt.plot()
```

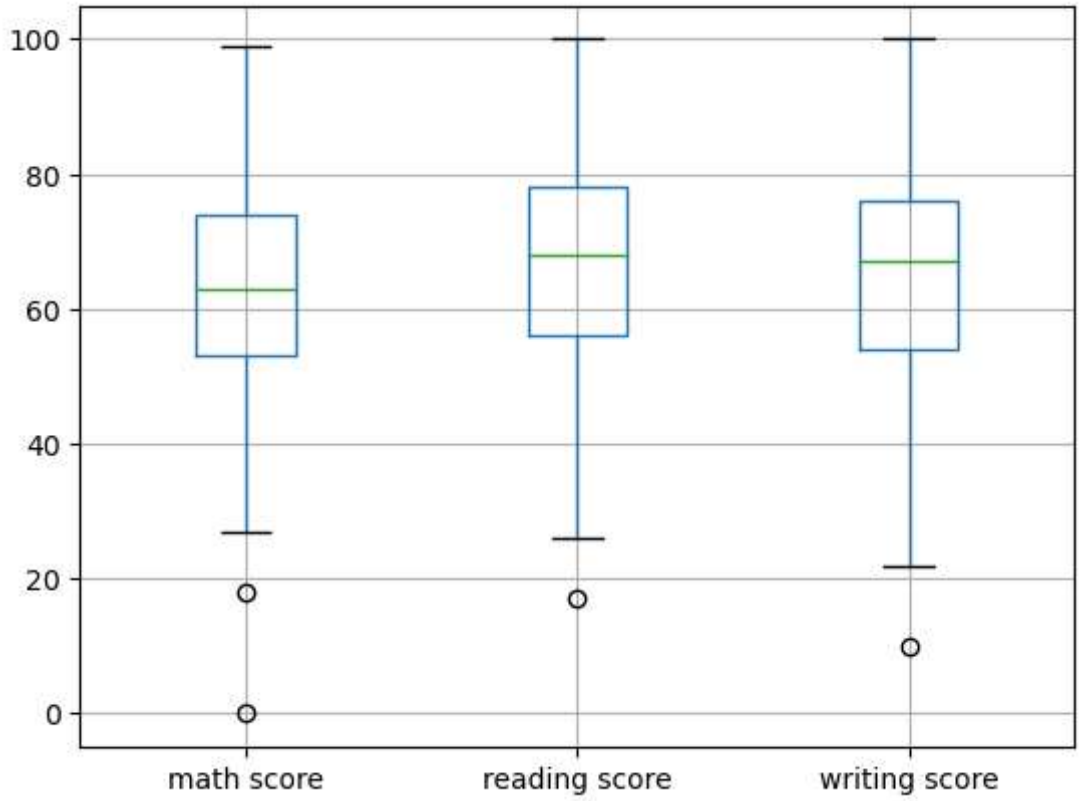
Out[201]:

```
[]
```



In [82]:
```
data.boxplot()
```

Out[82]:
```
<AxesSubplot:>
```

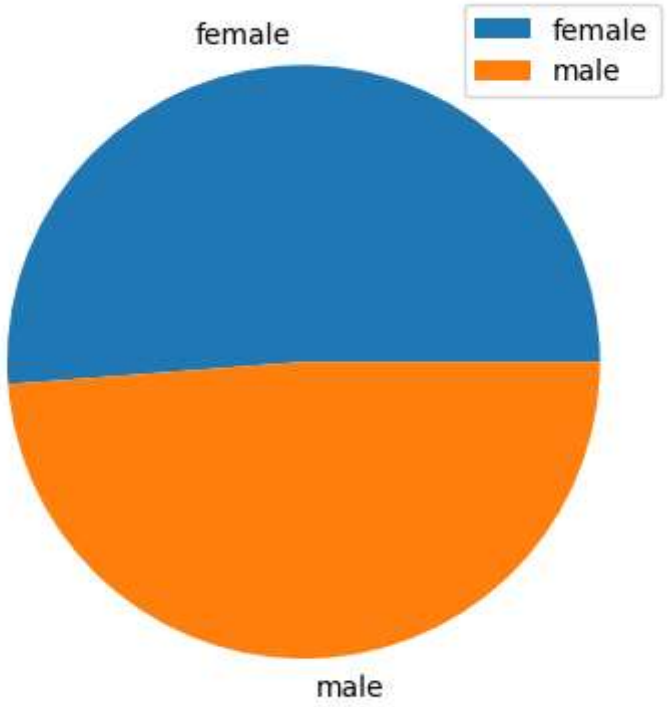


In [207…
```
x=data["gender"]
count=x.value_counts()
count
```

Out[207]:
```
female    64
male      61
Name: gender, dtype: int64
```

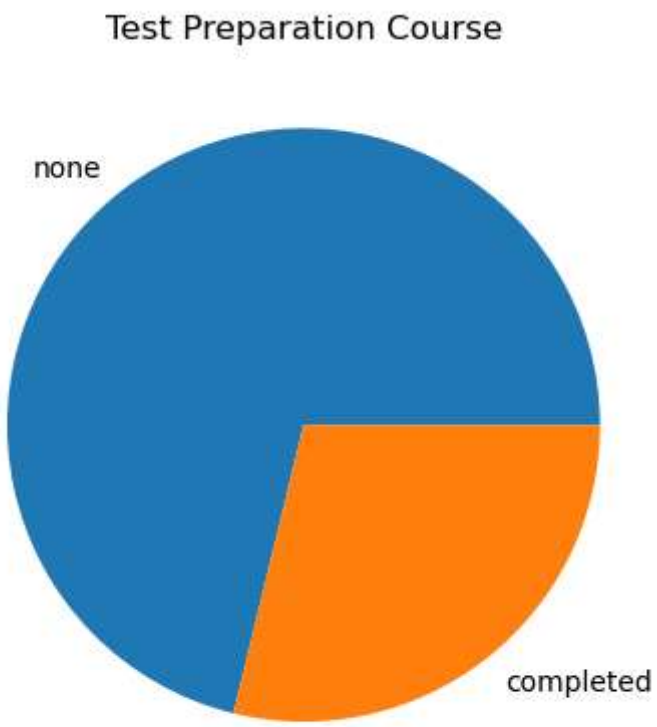
In [222…
```
o=count.keys()
o
```

In [225…
```
plt.pie(count,labels=o)
plt.legend()
plt.show()
```



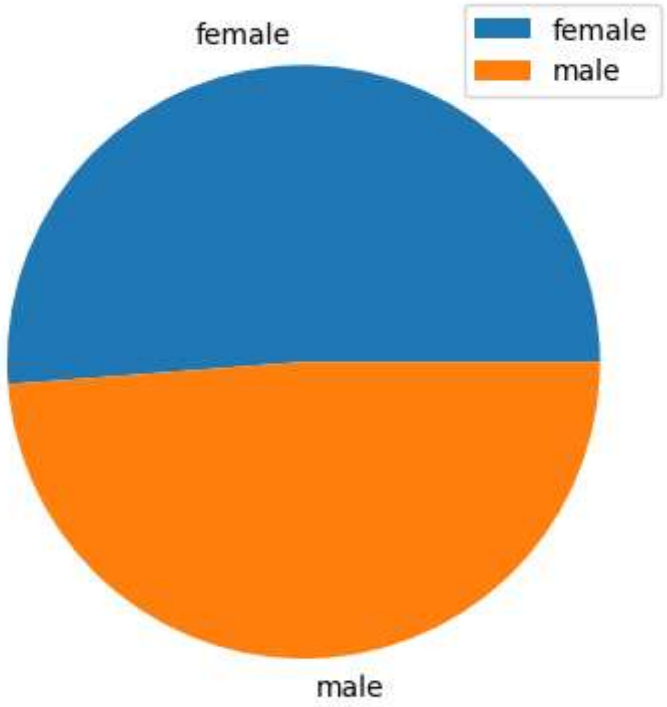
In [213…

```
import matplotlib.pyplot as plt

# Count occurrences of each category in the "test preparation course" column
x = data["test preparation course"].value_counts()

# Create a pie chart
plt.pie(x,labels=x.keys())

# Add title
plt.title('Test Preparation Course')

# Show the plot
plt.show()
```



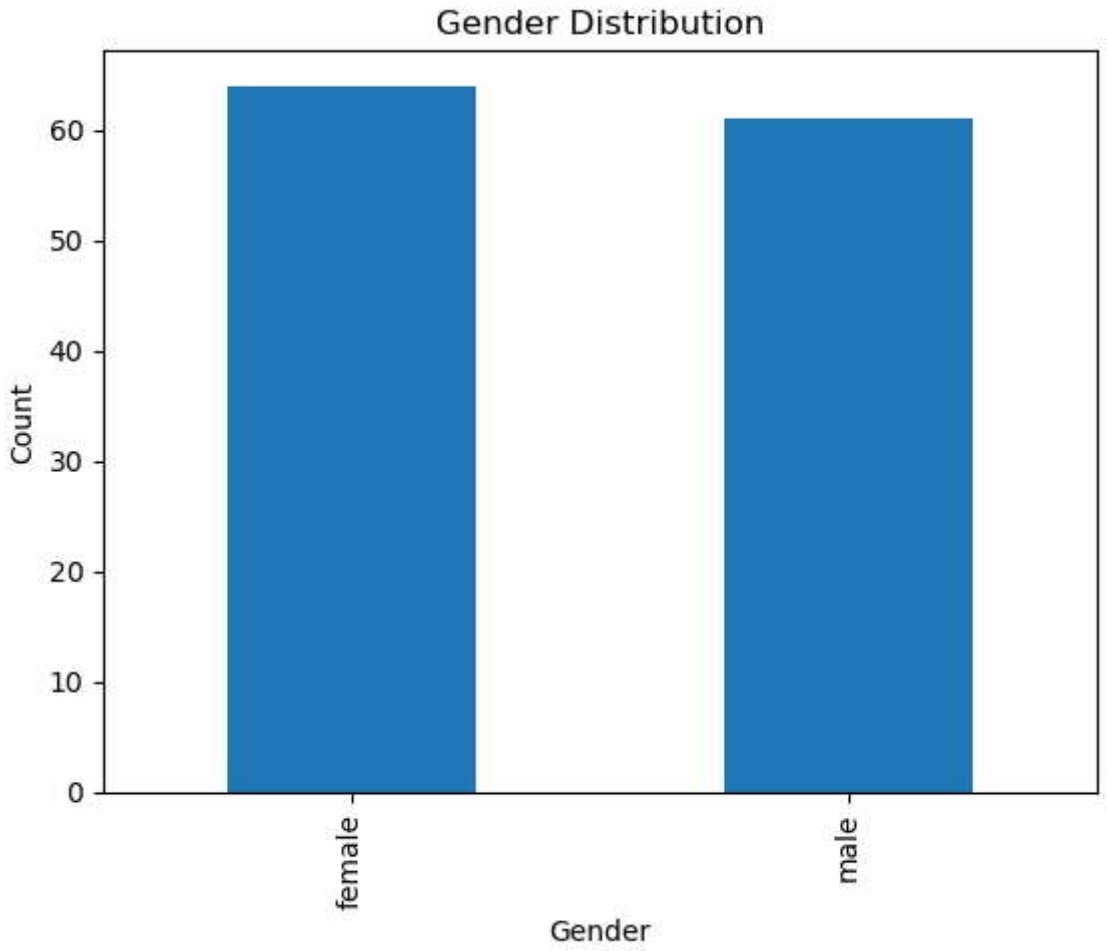
In [220…

```
x=data["gender"]
values=x.value_counts()

plt.pie(values,labels=values.keys())
plt.legend()
plt.show()
```



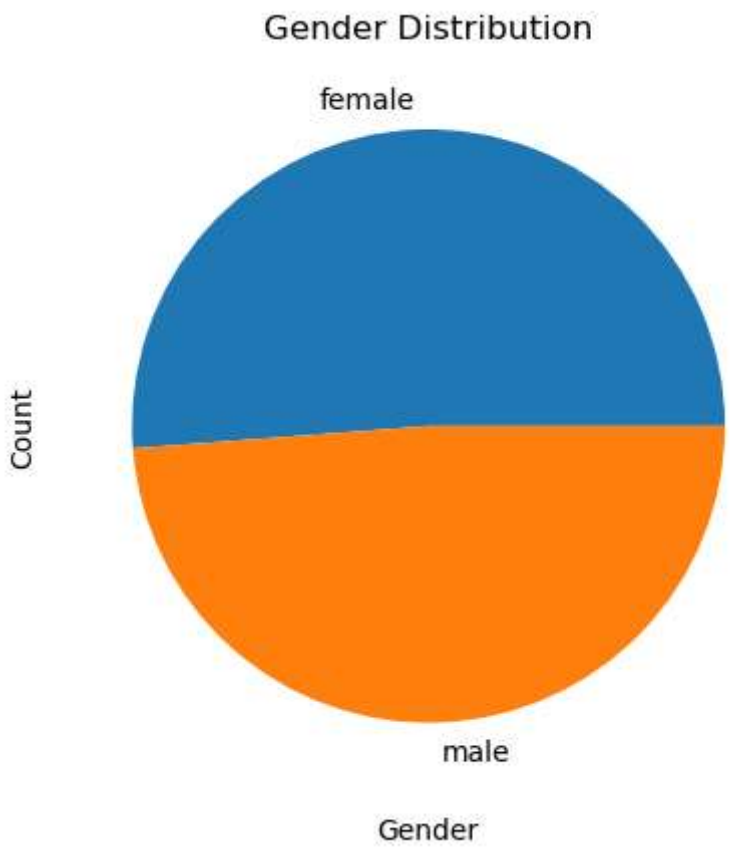
In [227…

```
import matplotlib.pyplot as plt

gender_counts = data['gender'].value_counts()

# Plotting
gender_counts.plot(kind='bar')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.title('Gender Distribution')
plt.show()
```



In [228…

```
import matplotlib.pyplot as plt

gender_counts = data['gender'].value_counts()

# Plotting
gender_counts.plot(kind='pie')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.title('Gender Distribution')
plt.show()
```



In [79]:
```
data["gender"].value_counts()
```

Out[79]:
```
female    64
male      61
Name: gender, dtype: int64
```

In [ ]: